SIMATS ENGINEERING

Short Answer Test

COURSE NAME: Software Testing **COURSE CODE: CSA3712**

COURSE OUTCOME COVERED

CO1: *Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.*

Assessment Tool Weightage: 30%

Sl. No	Questions	Bloom's Level
1	A startup team relies on random testing methods, leading to inconsistent results and missed defects. The project manager wants to improve testing effectiveness. As a test engineer, apply the concept of testing as an engineering activity to redesign the approach. Explain how structured test design improves quality and defect detection.	3
2	A banking system undergoes multiple failures due to lack of a defined testing process. The organization plans to standardize its testing activities. Illustrate how a well-defined testing process improves software quality in such real-time systems. Explain the benefits with relevant examples.	3
3	During development of an e-commerce platform, testing activities are performed inconsistently across phases. This leads to integration and system-level defects. Demonstrate how testing should be carried out as a structured process in SDLC. Explain how process discipline ensures better quality outcomes.	3
4	A login system shows mismatch between expected and actual outputs during validation. The development team is unsure whether the issue is an error, defect, or failure. Apply basic testing definitions to classify the issue. Explain the relationship between these terms with suitable examples.	3
5	A team struggles to detect defects early, leading to increased cost during later stages. The manager wants to improve defect detection efficiency. Apply a software testing principle to address this issue. Explain how the principle improves testing effectiveness with an example.	3
6	During requirement analysis of a healthcare system, several ambiguities go unnoticed. Later, these lead to defects in implementation. Show how a tester can contribute during the requirement phase to improve quality. Explain the importance of early testing involvement.	3
7	A payroll system produces incorrect outputs due to unclear and ambiguous requirements. The development team needs to identify the origin of defects. Analyze how such defects originate in the software lifecycle. Explain how requirement clarity can prevent such issues.	3

8	A system crashes when boundary input values are entered by users. The team suspects improper testing coverage. Classify this defect and justify your answer. Explain how boundary value analysis could have prevented this issue.	3
9	A company maintains a defect repository but rarely uses it for improving testing practices. This results in repeated defects across releases. Demonstrate how a defect repository can support better test case design. Explain its role in improving software quality.	3
10	An ATM system fails during withdrawal due to incorrect handling of transaction logic. The issue was not detected during testing. Apply your testing knowledge to identify and describe this as a functional defect. Explain how it could have been prevented.	3
11	A large application shows that most defects are concentrated in a few modules. The testing team wants to optimize their efforts. Use the concept of defect clustering to identify testing priorities. Explain how this improves testing efficiency.	3
12	A web application experiences slow performance under heavy user load. The issue needs to be categorized properly. Apply defect classification techniques to identify this type of defect. Explain how classification helps in better resolution.	3
13	A team is preparing for regression testing after multiple releases. They have access to historical defect data. Show how this data can be reused effectively for regression testing. Explain how it improves test coverage and efficiency.	3
14	During integration testing, multiple defects are found due to poor communication between developers and testers. The organization wants to improve collaboration. Identify the role of a tester in minimizing such defects. Explain how early involvement helps.	3
15	A software product repeatedly faces similar defects in every release cycle. The team needs a long-term solution. Apply defect prevention techniques to address recurring issues. Explain how prevention improves overall software quality.	3
16	A mobile application frequently crashes due to inconsistent testing approaches. The testing team wants to standardize practices. Apply testing principles to redesign the testing strategy. Explain how structured approaches improve reliability.	3
17	A company confuses debugging with testing, leading to delays in defect resolution. The team needs clarity on both concepts. Differentiate testing and debugging using a real-time scenario. Explain their roles in improving software quality.	3
18	An online shopping system behaves differently across browsers. This leads to customer dissatisfaction. Apply testing principles to design a cross-browser testing strategy. Explain how such issues can be effectively handled.	3

19	A logistics application produces inconsistent outputs due to improper validation. The issue is not detected early. Apply testing concepts to identify gaps in validation. Explain how proper testing improves system reliability.	3
20	A team skips structured test case design and directly executes random tests. This leads to poor defect detection. Apply testing as an engineering activity to improve the process. Explain the importance of systematic test design.	3
21	A healthcare system shows data inconsistency between modules due to integration issues. The testing team needs to analyze the problem. Apply testing principles to identify root causes. Explain how proper testing ensures data consistency.	3
22	A project experiences delays because testers are not involved early in development. The organization wants to improve collaboration. Explain the role of testers across SDLC phases. Show how early involvement improves quality.	3
23	A system fails during peak usage due to untested scenarios. The testing team missed critical cases. Apply testing principles to identify gaps. Explain how better test coverage prevents such failures.	3
24	A software team lacks proper defect documentation, making tracking difficult. The organization wants to improve defect management. Apply defect repository concepts to solve this issue. Explain how documentation enhances tracking and analysis.	3
25	A banking system produces repeated transaction failures due to improper testing practices. The team needs to improve quality. Apply software testing principles to redesign the testing approach. Explain how structured testing enhances reliability.	3
26	A system shows failures due to missing edge case testing. The testers focused only on normal inputs. Apply testing principles to include edge cases. Explain how boundary testing improves defect detection.	3
27	A development team struggles to maintain consistency in testing across modules. This leads to uneven quality. Apply process-based testing approaches to address this issue. Explain how standardization improves outcomes.	3
28	A web application fails due to improper handling of unexpected inputs. The testing team missed validation scenarios. Apply testing concepts to improve input validation. Explain how proper testing ensures robustness.	3
29	A software company faces frequent defect leakage into production. The management wants to reduce such issues. Apply testing principles to analyze the root cause. Explain how improved testing practices prevent leakage.	3
30	A project team is unable to identify defects early due to poor testing strategies. This increases overall cost and effort. Apply testing principles to redesign the strategy. Explain how early defect detection improves efficiency.	3

Q1. Testing as an Engineering Activity

Testing should be treated as a systematic engineering activity rather than random execution. The team should:

- Analyze requirements and identify test conditions.
- Design structured test cases using techniques such as equivalence partitioning and boundary value analysis.
- Define expected results, execute tests, record defects, and perform regression testing.

Q2. Well-Defined Testing Process

A defined testing process includes **planning, test design, test execution, defect reporting, retesting, and regression testing**.

For a banking system, requirements such as withdrawal limits, balance validation, and transaction security can be converted into test cases.

Q3. Testing as a Structured SDLC Process

Testing should be performed throughout the SDLC:

1. **Requirements:** Review requirements and identify ambiguities.
2. **Design:** Prepare test conditions and scenarios.
3. **Development:** Perform unit testing.
4. **Integration:** Test interactions between modules.
5. **System testing:** Test the complete application.
6. **Maintenance:** Perform regression testing.

Q4. Error, Defect and Failure

- **Error:** A human mistake made during development.
- **Defect:** A flaw in the software caused by an error.
- **Failure:** The incorrect behavior observed when the software executes.

Example: A developer misunderstands the password rule → writes incorrect login logic (**error**) → incorrect validation code (**defect**) → user with a valid password cannot log in (**failure**).

Q5. Early Defect Detection

The principle **"Testing should start early"** should be applied. Testers should review requirements and designs before coding begins.

For example, identifying an incorrect banking requirement during requirement review is much cheaper than discovering it after deployment. Early testing reduces rework, cost, and effort.

Q6. Tester During Requirement Phase

A tester can review requirements for:

- Ambiguity and missing information.
- Incorrect or conflicting requirements.
- Testability and completeness.

For a healthcare system, unclear requirements about patient records can be identified before implementation. Early tester involvement prevents defects and reduces correction cost.

Q7. Defects from Unclear Requirements

Unclear requirements can cause developers to interpret the same requirement differently. This leads to incorrect design, coding defects, and incorrect outputs.

Clear requirements should be **specific, complete, consistent, unambiguous, and testable**. Requirement reviews and stakeholder discussions can prevent such defects.

Q8. Boundary Value Analysis

A crash at boundary values indicates a defect related to **boundary/edge-case handling**, often caused by inadequate test coverage.

Boundary Value Analysis tests values at and around limits, such as:

- Minimum value
- Minimum + 1
- Maximum − 1
- Maximum value
- Maximum + 1

For example, if age allowed is 18–60, test **17, 18, 19, 59, 60 and 61**.

Q9. Defect Repository

A defect repository stores information such as defect description, cause, severity, affected module, and resolution.

Testers can analyze historical defects and create additional test cases for areas where defects repeatedly occurred. This prevents recurrence and improves test coverage and software quality.

Q10. Functional Defect in ATM

If an ATM incorrectly processes a withdrawal, such as deducting money without dispensing cash, it is a **functional defect** because the system does not perform the required function correctly.

It could be prevented through functional, integration, boundary, and transaction-flow testing with positive and negative test cases.

Q11. Defect Clustering

Defect clustering means that a small number of modules often contain a large proportion of defects.

The testing team should prioritize modules with high defect history and complexity. More testing effort can then be allocated to these modules, improving defect detection while using resources efficiently.

Q12. Performance Defect

Slow response under heavy user load is a **performance defect/non-functional defect**.

Performance testing such as load, stress, and endurance testing can identify the problem. Classification helps the team assign the issue to the appropriate performance analysis and resolution process.

Q13. Historical Defect Data in Regression Testing

Historical defect data can be used to identify previously problematic modules and create regression test cases for them.

After every release, these tests can be executed again to ensure old functionality still works. This improves coverage, reduces repeated defects, and saves test-design effort.

Q14. Role of Tester in Integration

Testers should participate early, understand interfaces between modules, review integration requirements, and communicate risks with developers.

Early collaboration helps identify interface mismatches and data-flow problems before integration testing. This reduces defects caused by poor communication.

Q15. Defect Prevention

For recurring defects, the team should perform **root-cause analysis** instead of only fixing individual defects. The team can conduct reviews, improve requirements, coding standards, test cases, and developer training. Prevention removes the underlying cause and reduces repeated defects in future releases.

Q16. Standardized Testing

The mobile application should follow a common testing process across all modules:

- Standard test plan and test-case format.
- Defined testing techniques.
- Consistent defect reporting.
- Regression testing after changes.

Standardization makes testing repeatable and improves application reliability.

Q17. Testing vs Debugging

Testing	Debugging
Finds defects/failures	Finds the cause and fixes the defect
Usually performed by testers	Usually performed by developers
Checks whether software behaves as expected	Locates and corrects faulty code

Example: Testing identifies that ATM withdrawal gives an incorrect balance. Debugging investigates the transaction code and fixes the calculation.

Q18. Cross-Browser Testing

The web application should be tested on supported browsers such as Chrome, Firefox, Edge, and Safari. Testers should check functionality, layout, compatibility, performance, and responsive behavior on different browsers and versions. Browser-specific defects should be logged and corrected before release.

Q19. Validation Gaps

Improper validation means that input or output conditions are not adequately checked.

The tester should identify missing validation scenarios and create positive, negative, boundary, and invalid-input test cases. Proper validation ensures consistent outputs and improves system reliability.

Instead of random testing, the team should:

- Analyze requirements.
- Identify test conditions.
- Design test cases systematically.

- Define expected results.
- Execute and document tests.
- Perform regression testing.

Systematic design improves test coverage, repeatability, and defect detection.

Q21. Integration and Data Consistency

The tester should identify interfaces between healthcare modules and verify data passed between them. Integration tests should check data formats, database updates, API responses, and synchronization. This helps identify interface defects and ensures consistent data across modules.

Q22. Tester Across SDLC

A tester contributes throughout the SDLC:

- **Requirements:** Review and identify ambiguities.
- **Design:** Identify testability issues.
- **Development:** Prepare and execute test cases.
- **Integration:** Test module interactions.
- **System testing:** Validate complete functionality.
- **Maintenance:** Perform regression testing.

Early involvement prevents defects and improves collaboration.

Q23. Test Coverage

The team should identify critical scenarios using requirements, risk analysis, usage patterns, boundary conditions, and historical defects.

For peak usage, load and stress scenarios should be included. Better test coverage increases the probability of finding critical defects before production.

Q24. Defect Repository

Every defect should be documented with:

- Defect ID
- Description
- Steps to reproduce
- Severity and priority
- Affected module
- Status
- Root cause and resolution

Q25. Structured Testing in Banking

The banking system should use a planned testing process covering requirements, functional testing, integration testing, security-related scenarios, boundary conditions, and regression testing.

Structured testing ensures important transaction scenarios are consistently tested and improves reliability by reducing missed defects.

Q26. Edge Case and Boundary Testing

Testing only normal inputs can miss important defects. Testers should include minimum, maximum, just-below, and just-above boundary values.

For example, for a transaction limit of ₹10,000, test values such as **₹9,999, ₹10,000, and ₹10,001**. Boundary testing helps detect failures at limits.

Q27. Process-Based Testing

A common testing process should be followed across all modules, including standard test plans, test-case formats, execution procedures, defect reporting, and regression testing.

Standardization reduces variation between teams and ensures consistent quality across modules.

Q28. Input Validation

The application should test valid, invalid, empty, unexpected, boundary, and special-character inputs.

For example, a registration form should reject invalid email formats and unexpected values. Proper input validation prevents crashes, incorrect processing, and security-related problems, improving robustness.

Q29. Defect Leakage

Frequent production defects indicate gaps in requirements, test coverage, test execution, or regression testing.

The team should perform root-cause analysis, strengthen test cases, increase coverage, conduct reviews, and perform regression testing. These practices reduce defect leakage into production.

Q30. Early Defect Detection

The team should introduce testing early in the SDLC through requirement reviews, design reviews, unit testing, integration testing, and continuous regression testing.

Finding a defect during requirements or development is generally easier and less costly to fix than finding it after deployment. Therefore, early detection reduces rework, cost, and effort while improving quality.

Code of Conduct:

*I **Haritha D / 192421282** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: *Haritha*

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	**30**	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	**25**	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	**20**	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	**15**	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	**10**	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand